Design for Social Innovation (DSI) · Individual Submission

Post-Demo Feedback Summary

Project 01B: Finding the Next Best Step

Team DSIders

Swayam Goyal (2024111012)
Partner organisation: The Barabari Collective

21 October 2026

This document summarises the feedback I took away from our project demo, from the TAs, the professors and our partner NGO, The Barabari Collective. I have picked the four points that I think matter most for the project. For each one I explain the feedback in my own words, decide whether it needs immediate refinement or belongs to future scope, and set out how I would act on it.

#	Feedback	Category
1	Track multiple applications under one mentor	Immediate refinement
2	Separate parsing of candidates and jobs, with two-way matching	Future scope
3	Explain why a step is recommended ("Why this match?")	Immediate refinement
4	Re-assign the mentor when a student changes path	Immediate refinement

1. Tracking multiple applications under one mentor

Feedback overview

When a student applies to several openings, these applications should not be treated as separate, unconnected items. Each one should be linked to the student's assigned mentor, so that the caseworker sees all of a student's active applications in one place instead of tracking them in offline spreadsheets. I read this as a sign that Barabari sees our app as part of its casework, not only as a guidance tool. In Sprint 1 we kept applications out of scope, but the demo made it clear that they are part of the casework the app needs to support now.

Categorisation: immediate refinement

Barabari's caseworkers already handle several applications per student today, and they do it in spreadsheets outside the app. Our app already stores openings and assigned mentors, so a basic tracker is a small extension of what exists. Without it, the next step the app gives and the applications the student is actually making sit in two different places, and the caseworker has to join them by hand.

Actionable steps

1. Add an applications table that links a student, an opening, the assigned mentor and a status (for example applied, interview, offer or closed).
2. List a student's active applications on the caseworker's student detail screen, and show a count of applications that need follow-up on the *My students* list.
3. Let the student mark an application from the job openings screen, so the record starts where the student acts.

4. Check the status names against what Barabari tracks today, ideally using an anonymised sample of their current sheet, and adjust them before the pilot.
5. Build this together with item 4, so that applications always follow the correct mentor.

2. Separate parsing of candidates and jobs, with two-way matching

Feedback overview

Our current intake asks for the preferred role, resume and skills up front, and then builds one fixed path towards Barabari's listings. The feedback asks us to parse the candidate's profile and the job descriptions separately into one common structure (qualifications, skills, certifications and documents), and then match them to recommend the best opportunities, each with a roadmap for closing the gap. In my view, our engine assumes the student already knows their goal, which was true of the user we designed for in Sprint 1. The feedback points out that many students do not, and that some openings suit students who would never have named them. The question changes from "How do I reach this role?" to "Which roles fit me now, and what is missing for each?"

Categorisation: future scope

This is the largest change of the four. It needs a new parser for job descriptions, a shared schema, and a ranking step, and it touches S1, S3 and the data model. It also depends on a set of real job descriptions from Barabari.

Actionable steps

1. Define one shared schema for qualifications, skills with levels, certifications and documents, used for both student profiles and jobs.
2. Extend S1 with a job-description mode that parses openings into this schema. As with AI-drafted roles today, a caseworker reviews the parsed requirements before they are used.
3. Run S3 against each opening and rank openings by how many requirements are already met, weighted by how quickly the remaining gaps can be closed. Show the top three, each with its own next step and a "Why this match?" card from item 3.
4. Keep the language model out of the ranking, so the matching stays deterministic and explainable, in line with our first design principle.
5. With Barabari caseworkers, compare our top three for each test persona against the openings they would choose themselves.

3. Explaining why a step is recommended

Feedback overview

Reviewers felt that the recommended steps behave like a "black box". The student is told what to do next, but not why that path was chosen over another. In my reading, the reasons already exist inside our engine, since the gap reasoner (S3) attaches a reason to every row. The problem is that the student never sees those reasons together at the moment of decision, so the logic is hidden exactly where trust matters most.

Categorisation: immediate refinement

This affects trust in the app's main output, and the information needed is already produced by the engine. It is a small change with a large effect on how believable the recommendation feels.

Actionable steps

1. Gather S3's reasons into a "Why this match?" card on the *Your next step* screen. The card shows the criteria the student already meets, the exact prerequisite that is missing, and why this step comes first, for example because it unlocks the steps after it.
2. Show where each matched fact came from (the student, their resume or a caseworker), using the evidence and source already stored for each requirement.
3. Add a test that fails if any recommendation is produced without a complete card, extending the explainability check in our evaluation criteria.
4. Try the wording with two or three students and one Barabari caseworker to check that it is plain enough to follow.

4. Re-assigning the mentor when a student changes path

Feedback overview

The link between a student and a mentor should not be fixed. If a student changes direction, for example from digital data entry to retail management or hardware repair, the mentor should change as well, based on domain, the mentor's specialisation or the geographic area. In our app the mentor is attached to the student. After a change of path, the student can be left with someone who does not know the new field, which weakens the handover we rely on whenever the app is unsure.

Categorisation: immediate refinement

This directly affects whether a handover reaches the right person. The change is also small, because each caseworker record already stores their topics, language, centre and availability.

Actionable steps

1. Move the mentor assignment from the student record to the student's active goal, so that it belongs to the path rather than the person.
2. Write a matching rule that compares the goal's role domain with each mentor's topics, then prefers the same centre, then a shared language, then availability. If no one matches, it falls back to any available mentor.
3. When the goal changes, re-run the rule automatically and leave a hand-off note in the student's thread, so the new mentor knows the history and the previous mentor can see the change.
4. Add tests for a change of path, a case with no matching mentor, and a matching mentor who is away.

Closing note

Three of the four points are immediate refinements. They strengthen what the app already does: tracking what a student is actually doing, explaining each recommendation, and sending the student to the right person. Two-way matching is future scope, because it widens the app from guiding a student along one chosen path to supporting their whole search for work. I would start with mentor re-assignment and application tracking together, since applications must follow the right mentor. The "Why this match?" card would come next, and two-way matching after that, because it reuses the card.